



Carrera:
Programación y WebMaster

Docente:
Mario ALonso Segovia Gutierrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Manuel David Castro Blanco
Pablo Leonel Salomón Campos
Bryant Maximiliano Dzul Perez
Gerardo Alexander Díaz León

Fecha:
07 de Agosto 2026

Índice

Índice	2
Propósito del documento.	6
Alcance del sistema.	6
Objetivos del proyecto.	6
Descripción general del problema.	6
Descripción general del Sistema	7
Perspectiva general del sistema.	7
Funcionalidades principales.	7
Tipos de usuarios.	8
Restricciones generales.	8
Dependencias externas.	9
Stakeholders y actores	9
Stakeholders principales.	9
Actores del sistema, intereses y responsabilidades.	10
Requerimientos funcionales	11
Requerimientos no funcionales	12
Casos de uso	13
Diagrama de Casos de uso	22
Reglas de negocio	22
¿Qué es nuestra Definition of Done?	43
Criterios (una funcionalidad está Done cuando...)	43
Notas de aplicación	45
Compromiso del equipo	45
Propósito de este documento	45
Regla 1 — Nombres descriptivos y en el lenguaje del dominio	45
Regla 2 — La lógica de negocio vive en Servicios de dominio, no en los controladores	46
Regla 3 — Nada de números ni cadenas “mágicas”: usar constantes con nombre	47
Regla 4 — Validación y autorización fuera del controlador (FormRequest / Policy)	47
Regla 5 — Comentarios que explican el por qué, y PHPDoc con tipos	48
Convenciones de estilo complementarias	48
Compromiso del equipo	49
Introducción	49
Descripción del Proyecto Nombre del proyecto	49
Problema que resuelve	49
Tecnologías utilizadas	50
Arquitectura general del sistema	51
Módulos principales	51

Estrategia General de Pruebas	54
Tipos de pruebas	54
Momento de ejecución	54
Responsables	55
Herramientas para documentar y registrar resultados	55
Justificación	57
Plan de Pruebas Unitarias	57
PU-01: Cálculo de fecha de egreso prevista	57
PU-02: Generación automática de folio de expediente	57
PU-03: Verificación de permisos por rol (RBAC)	58
PU-04: Cálculo de stock disponible de medicamento por lotes	59
PU-05: Validación de límite de permanencia (6 meses máximo)	60
PU-06: Validación de administración de medicamento con stock insuficiente	60
PU-07: Búsqueda difusa de personas por trigram (prevención de duplicados)	61
Plan de Pruebas de Integración	62
Plan de Pruebas End-to-End (E2E)	68
E2E-01: Admisión completa de un nuevo residente	68
E2E-02: Proceso clínico interdisciplinario y evaluación trimestral	70
E2E-03: Egreso de residente con cierre de expediente	71
E2E-04: Registro de bitácora diaria con servicios e incidencia	73
Matriz de Cobertura de Pruebas	75
Criterios de Aceptación	78
Funcionamiento correcto	78
Cumplimiento del requerimiento	78
Manejo adecuado de errores	79
Validación de datos	79
Integración correcta con otros módulos	79
Ausencia de errores críticos	80
Control de acceso	80
Riesgos y Limitaciones	80

Introducción

El Albergue Mixto "Ajo'olnáj — Tu Techo", dependencia de la Secretaría de Inclusión del Gobierno del Estado de Campeche, es una institución dedicada a brindar atención integral, temporal y digna a personas adultas en situación de calle, abandono social, exclusión, movilidad humana o carencia de redes de apoyo. Actualmente, los procesos operativos, clínicos y administrativos de la institución se gestionan mediante documentación en formato papel y herramientas de ofimática desintegradas, lo que genera pérdida de información, dificultad en el seguimiento integral de los residentes, falta de trazabilidad en movimientos y ausencia de alertas automáticas para el cumplimiento de protocolos institucionales.

El presente documento integra la especificación completa del **Sistema de Gestión Institucional del Albergue "Ajo'olnáj — Tu Techo"**, un proyecto de desarrollo de software que busca centralizar y automatizar todos los procesos relacionados con la admisión, atención interdisciplinaria, seguimiento clínico, control de estancias, gestión de inventario y egreso de las personas usuarias. La solución tecnológica está siendo desarrollada por un equipo multidisciplinario bajo una combinación de metodologías Cascada y Kanban, implementada con tecnologías modernas (Laravel 12, PostgreSQL, Docker) y una arquitectura modular basada en Domain-Driven Design.

Este documento reúne el análisis de requerimientos, la planificación del proyecto, las reglas de calidad del código, la estrategia de pruebas y los criterios de aceptación que garantizarán un sistema funcional, seguro, mantenible y alineado con los objetivos institucionales de dignidad humana, inclusión social y reinserción de las personas atendidas.

Propósito del documento.

El propósito del documento es orientar la operación, organización interna y prestación de servicios del Albergue Mixto “A Jó’olnaj – Tu Techo”, asegurando que las acciones del personal se desarrollen conforme a los principios de dignidad humana, inclusión social, orden institucional y reinserción social.

Alcance del sistema.

El sistema aplica de manera obligatoria al personal directivo, administrativo y operativo del albergue, así como a las áreas de la Secretaría de Inclusión relacionadas con la planeación, supervisión y evaluación de los servicios. También incluye a toda persona que intervenga en la atención, acompañamiento o canalización de las personas usuarias.

Objetivos del proyecto.

El objetivo general es brindar atención integral, temporal y digna a personas adultas en situación de calle, abandono social o vulnerabilidad, mediante servicios de alojamiento, alimentación y actividades formativas orientadas al fortalecimiento personal y a la reinserción social o familiar.

Los objetivos específicos incluyen proporcionar un espacio seguro y libre de violencia, garantizar alimentación, higiene y alojamiento, promover la sana convivencia, implementar actividades educativas y ocupacionales, establecer mecanismos de canalización y fomentar un ambiente institucional de respeto, orden y disciplina.

Descripción general del problema.

El problema principal que atiende el sistema es la situación de vulnerabilidad social de personas adultas que se encuentran en condición de calle, abandono social, exclusión, movilidad humana o carencia de redes de apoyo. Estas personas requieren atención

temporal, acompañamiento interdisciplinario, servicios básicos y mecanismos de reinserción familiar o social.

Descripción general del Sistema

Perspectiva general del sistema.

El Albergue Mixto “A Jó’olnaj – Tu Techo” funciona como una casa de transición para personas en situación de calle. Su operación está basada en procedimientos normativos, administrativos e interdisciplinarios que regulan el ingreso, atención primaria, diagnóstico integral, seguimiento mensual, evaluación a tres meses y egreso de las personas usuarias.

Funcionalidades principales.

Las funcionalidades principales del sistema son:

1. Recepción e ingreso de personas solicitantes.
2. Entrevista inicial y detección de vulnerabilidad.
3. Verificación de criterios de elegibilidad.
4. Firma del Código de Conducta.
5. Apertura de expediente o Informe Clínico Unificado.
6. Asignación de espacio y registro de ingreso.
7. Atención primaria durante las primeras 72 horas.
8. Diagnóstico social, médico y psicológico.
9. Elaboración del Plan Individual de Atención.
10. Seguimiento mensual interdisciplinario.
11. Evaluación trimestral para alta o permanencia.
12. Procedimiento de egreso y cierre de expediente.

Tipos de usuarios.

Los usuarios principales del sistema son personas adultas de entre 18 y 60 años que se encuentren en situación de calle, abandono social, reinserción familiar o social, exclusión, movilidad humana o carencia de redes de apoyo. La permanencia debe ser voluntaria y condicionada al cumplimiento del Reglamento Interno y Código de Conducta.

También pueden considerarse usuarios internos del sistema el personal directivo, operativo y técnico que participa en la atención y seguimiento de los casos.

Restricciones generales.

Las principales restricciones del sistema son:

- La persona debe aceptar voluntariamente su ingreso y permanencia.
- La estancia mínima es de 5 días en casos aplicables.
- La estancia media es de 3 meses.
- La estancia máxima es de 6 meses, sujeta a evaluación interdisciplinaria.
- La permanencia depende del cumplimiento del Reglamento Interno y Código de Conducta.
- El albergue puede determinar el egreso por incumplimiento grave, condición de salud que exceda la capacidad institucional, voluntad del usuario, canalización o conclusión del apoyo temporal.

Dependencias externas.

El sistema depende de la Secretaría de Inclusión del Gobierno del Estado de Campeche, especialmente de la Dirección de Reinserción Social y Personas en Situación de Calle y la Coordinación del Albergue. También contempla la vinculación con instancias de salud, asistencia social, programas sociales, reinserción laboral,

reintegración familiar y otras dependencias competentes según la valoración interdisciplinaria de cada caso.

Stakeholders y actores

Stakeholders principales.

Los principales stakeholders son:

- Personas usuarias del albergue.
- Dirección o Coordinación del Albergue.
- Secretaría de Inclusión del Gobierno del Estado de Campeche.
- Dirección de Reinserción Social y Personas en Situación de Calle.
- Personal de Trabajo Social.
- Personal de Enfermería.
- Personal de Psicología.
- Encargados polivalentes de turno.

Actores del sistema, intereses y responsabilidades.

Actores del sistema

Intereses y responsabilidades · Albergue “A Jó’olnáj – Tu Techo”

Actor	Intereses	Responsabilidades
Dirección / Coordinación del Albergue	Garantizar el funcionamiento ordenado, seguro y normativo del albergue.	Autorizar ingresos, diagnósticos integrales, evaluaciones a 3 meses y egresos; supervisar el cumplimiento del manual, reglamento interno y código de conducta; resguardar expedientes y validar cierres administrativos.
Trabajo Social	Identificar la situación social de la persona usuaria y favorecer su reintegración familiar o social.	Realizar entrevista inicial, diagnóstico social, identificación de redes de apoyo, seguimiento del PIA, gestión documental, canalizaciones institucionales y apoyo en el egreso.
Enfermería	Proteger la salud física de las personas usuarias durante su estancia.	Realizar valoración física, tomar signos vitales, registrar antecedentes médicos, identificar enfermedades crónicas, controlar tratamientos, dar seguimiento a citas médicas y registrar evolución médica.
Psicología	Atender la salud emocional y fortalecer habilidades socioemocionales de las personas usuarias.	Realizar entrevista de primer contacto, detectar crisis emocionales o riesgos, brindar contención emocional, sesiones individuales o grupales, intervención en crisis y diagnóstico psicológico.
Encargados polivalentes de turno	Mantener la operación diaria, el orden y la seguridad del albergue en los distintos turnos.	Apoyar en la recepción de usuarios, registrar ingresos, informar incidencias, vigilar el cumplimiento de normas internas y acompañar la operación en turnos matutino, vespertino, nocturno y fines de semana.
Personas usuarias del albergue	Recibir alojamiento, alimentación, higiene, acompañamiento y atención integral temporal.	Aceptar voluntariamente el ingreso, cumplir el reglamento interno y el código de conducta, participar en su proceso de atención, respetar la convivencia y colaborar con las áreas interdisciplinarias.
Secretaría de Inclusión del Gobierno del Estado de Campeche	Asegurar que el programa responda a la atención de personas en situación de vulnerabilidad.	Diseñar, operar, supervisar y evaluar acciones dirigidas a personas en situación de calle; vincular áreas relacionadas con planeación, supervisión y evaluación del albergue.
Instancias externas de salud, asistencia social, reinserción laboral o familiar	Complementar la atención que el albergue no puede cubrir directamente.	Recibir canalizaciones, brindar servicios especializados y apoyar procesos de salud, asistencia social, reinserción laboral, reintegración familiar o creación de redes de apoyo.

Requerimientos funcionales

RF-01: El sistema deberá permitir el registro de personas beneficiarias/albergadas mediante un formulario digital.

RF-02: El sistema deberá permitir actualizar la información personal y social de las personas beneficiarias.

RF-03: El sistema deberá permitir consultar el expediente completo de una persona beneficiaria autorizada.

RF-04: El sistema deberá permitir registrar ingresos de personas al albergue indicando fecha y hora.

RF-05: El sistema deberá permitir registrar egresos de personas del albergue sin perder historial de movimientos.

RF-06: El sistema deberá permitir administrar la asignación de camas y espacios disponibles dentro del albergue.

RF-07: El sistema deberá permitir registrar los servicios brindados a las personas beneficiarias dentro del albergue.

RF-08: El sistema deberá permitir registrar y resguardar las pertenencias personales de las personas albergadas.

RF-09: El sistema deberá permitir consultar el historial de atenciones, movimientos y servicios de cada persona beneficiaria.

RF-10: El sistema deberá permitir registrar incidencias y eventos relevantes ocurridos dentro del albergue.

RF-11: El sistema deberá permitir registrar y dar seguimiento a procesos de reintegración social.

RF-12: El sistema deberá permitir registrar y dar seguimiento a procesos de reintegración social.

RF-13: El sistema deberá permitir cargar y almacenar documentos digitales y archivos adjuntos relacionados con los expedientes.

RF-14: El sistema deberá permitir generar reportes estadísticos y administrativos sobre la operación del albergue.

RF-15: El sistema deberá permitir generar alertas y notificaciones internas sobre eventos importantes o pendientes de atención.

RF-16: El sistema deberá permitir consultar información mediante funciones de búsqueda y filtrado de registros en tiempo real.

RF-17: El sistema deberá permitir registrar visitas, apoyos y seguimientos psicológicos o sociales realizados a las personas beneficiarias.

RF-18: El sistema deberá permitir registrar y monitorear el seguimiento médico básico y control de medicamentos, en caso de ser requerido.

RF-19: El sistema deberá permitir exportar información y reportes en formatos PDF y Excel.

RF-20: El sistema deberá permitir administrar usuarios y asignar roles específicos dentro del sistema.

Requerimientos no funcionales

RNF-01: El sistema deberá garantizar la protección y confidencialidad de los datos personales almacenados.

RNF-02: El sistema deberá implementar autenticación de usuarios mediante credenciales seguras de acceso.

RNF-03: El sistema deberá ser compatible con navegadores modernos como Google Chrome, Microsoft Edge y Mozilla Firefox.

RNF-04: El sistema deberá contar con una interfaz intuitiva, amigable y fácil de usar para los usuarios.

RNF-05: El sistema deberá realizar respaldos automáticos y permitir la recuperación de la información en caso de fallos.

RNF-06: El sistema deberá mantener una alta disponibilidad y estabilidad durante su operación.

RNF-07: El sistema deberá permitir escalabilidad para incorporar nuevas funcionalidades en el futuro.

RNF-08: El sistema deberá utilizar una base de datos segura, organizada y estructurada correctamente.

RNF-09: El sistema deberá registrar trazabilidad y auditoría de las acciones realizadas por los usuarios.

RNF-10: El sistema deberá cumplir con las normativas y políticas de protección de datos aplicables.

RNF-11: El sistema deberá ofrecer un rendimiento óptimo en consultas, búsquedas y generación de reportes.

RNF-12: El sistema deberá contar con un diseño responsive compatible con dispositivos móviles y computadoras.

RNF-13: El sistema deberá facilitar el mantenimiento y actualización del software sin afectar su funcionamiento.

RNF-14: El sistema deberá implementarse bajo una arquitectura centralizada y segura.

RNF-15: El sistema deberá gestionar permisos y accesos según el perfil y rol de cada usuario.

Casos de uso

Nombre	Apertura de Expediente
Objetivo	Crear el registro inicial en el sistema (Informe Clínico Unificado) para un nuevo usuario que solicita asilo.
Actor principal	Trabajo Social, Encargado de Turno
Precondiciones	La persona ha sido recibida en el albergue y se le ha realizado la entrevista inicial . El usuario ha aceptado y firmado el Código de Conducta.
Flujo principal	1. Trabajo Social busca en el sistema si la persona ya cuenta con un expediente previo. 2. Al no existir, selecciona la opción "Apertura de Expediente". 3. Ingresa los datos generales de la persona (nombre, edad, sexo,

	etc.). 4. El sistema genera un número de folio/expediente único. 5. El actor guarda el registro en la base de datos.
Flujos alternos	La persona no cumple con los criterios de elegibilidad: El proceso de apertura se detiene y se activa el flujo extendido de "Gestión de canalizaciones externas" .
Postcondiciones	Se crea el expediente electrónico del usuario con estado "Activo" y queda habilitado para que las demás áreas registren sus diagnósticos.

Nombre	Registro Diagnóstico Social
Objetivo	Documentar la situación social, familiar y problemáticas identificadas del usuario durante la ruta crítica de las primeras 72 horas .
Actor principal	Trabajo Social.
Precondiciones	El usuario debe tener un expediente previamente abierto en el sistema.

Flujo principal	1. El actor ingresa al sistema y abre el expediente del usuario. 2. Selecciona la sección correspondiente al "Diagnóstico Social". 3. Registra la situación social y familiar, así como las redes de apoyo identificadas. 4. Redacta las problemáticas sociales identificadas y la intervención inicial propuesta. 5. Guarda el registro, firmando electrónicamente la intervención.
Flujos alternos	El usuario requiere atención de otra institución de manera urgente: El actor utiliza el punto de extensión hacia "Gestión de canalizaciones externas" para referir al usuario a una instancia pertinente.
Postcondiciones	La Cédula Clínica Interdisciplinaria (CCI) se actualiza con la información del área social.

Nombre	Asignación de Espacio
Objetivo	Vincular al usuario con un espacio físico específico (cama/área) dentro del albergue para su pernocta temporal.
Actor principal	Trabajo Social, Encargado de Turno.

Precondiciones	Se ha verificado que la persona cumple los criterios de elegibilidad y se ha completado la apertura de su expediente.
Flujo principal	1. El actor ingresa al módulo de "Asignación de Espacio". 2. Selecciona el expediente del usuario recién ingresado. 3. El sistema muestra la disponibilidad actual de camas en el área de dormitorio o área transitoria. 4. El actor selecciona un espacio disponible y confirma la asignación. 5. El sistema registra el ingreso físico al albergue y actualiza el inventario de espacios.
Flujos alternos	No hay camas disponibles: El sistema emite una alerta de capacidad máxima. El actor debe ser canalizado a otro albergue.
Postcondiciones	El sistema marca la cama como "Ocupada" y asocia el espacio físico al número de expediente del usuario.

Nombre	Control de Tratamiento y Citas Médicas
Objetivo	Llevar un registro del historial de intervenciones, educación para la salud, administración de medicamentos y programación de citas médicas del usuario.

Actor principal	Enfermería.
Precondiciones	El usuario cuenta con un expediente abierto y un "Registro Diagnóstico Médico" previo donde se documentaron antecedentes y signos vitales.
Flujo principal	1. El personal de enfermería selecciona el expediente del usuario. 2. Ingresa al módulo de "Control de Tratamientos". 3. Registra la administración de un medicamento (consultando el stock del "Inventario de medicamentos") o agenda la fecha y hora de una próxima cita médica. 4. Redacta la nota de evolución médica u observaciones relevantes. 5. Guarda y actualiza la información en el sistema.
Flujos alternos	Medicamento no disponible: El sistema advierte que el medicamento indicado no cuenta con stock suficiente en el "Inventario de medicamentos". El actor debe gestionar el insumo antes de registrar la administración.
Postcondiciones	El historial médico del usuario queda actualizado, reflejando el seguimiento continuo mensual.

Nombre	Registro Diagnóstico Psicológico
---------------	---

Objetivo	Asentar la evaluación del estado emocional, cognitivo y el plan de intervención psicológica del usuario.
Actor principal	Psicología.
Precondiciones	Haber realizado la entrevista de primer contacto con el usuario.
Flujo principal	1. El profesional de psicología accede al expediente del usuario. 2. Selecciona el apartado de "Diagnóstico Psicológico". 3. Registra el estado emocional, cognitivo y el diagnóstico presuntivo. 4. Establece el plan de intervención (individual o grupal). 5. Guarda el registro en la Cédula Clínica Interdisciplinaria o Nota de Seguimiento.
Flujos alternos	Detección de crisis: Si el psicólogo detecta un riesgo inminente o crisis emocional, registra inmediatamente una "Contención emocional inmediata" como intervención prioritaria antes de finalizar el diagnóstico formal.
Postcondiciones	El diagnóstico psicológico queda integrado al Plan Individual de Atención (PIA) del usuario.

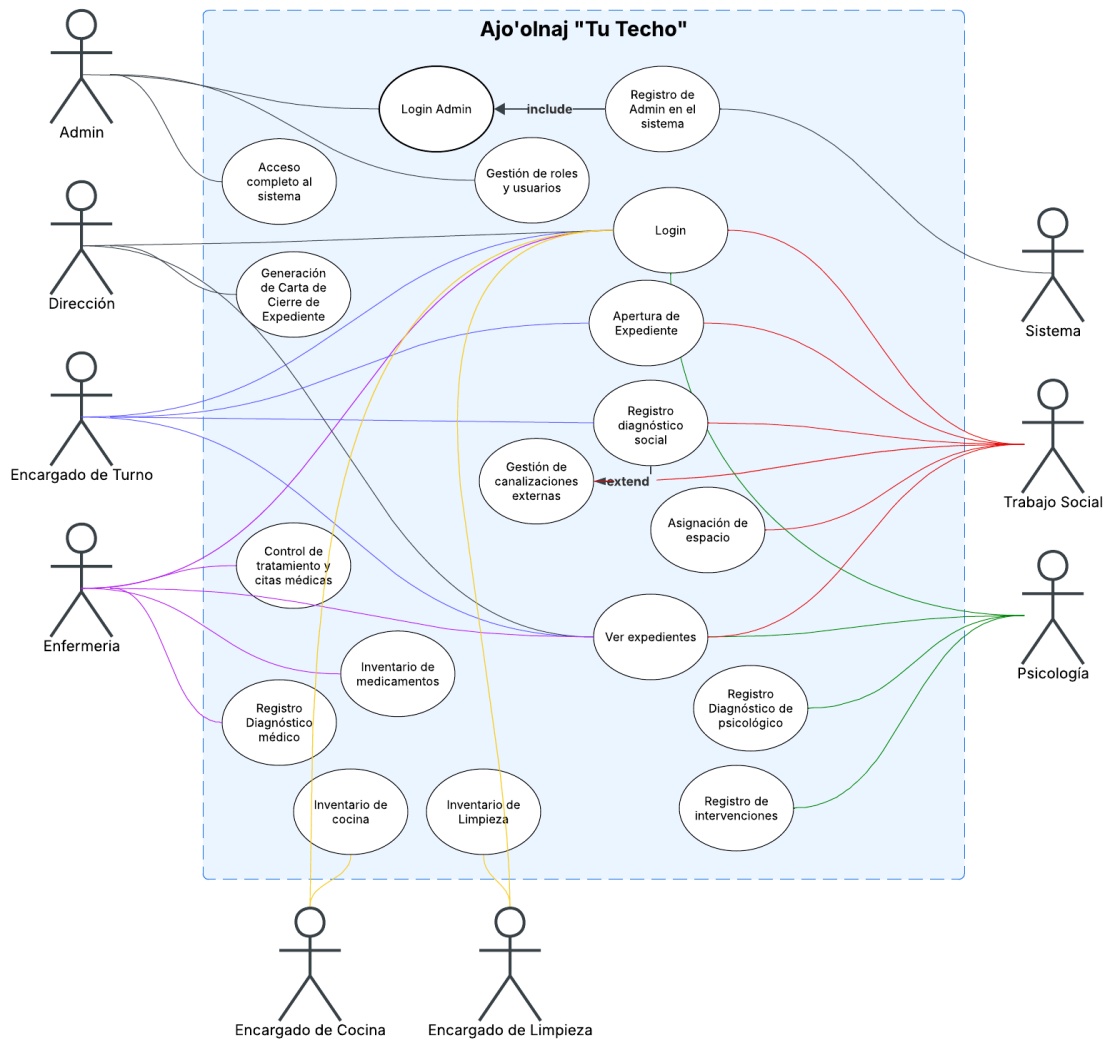
Nombre	Generación de Carta de Cierre de Expediente
Objetivo	Formalizar el egreso de una persona usuaria y el cierre definitivo de su expediente administrativo y clínico .
Actor principal	Dirección.
Precondiciones	Debe existir un "Informe Final Interdisciplinario (IFI)" completado y una causal válida de egreso (ej. voluntad del usuario, reintegración, conclusión del apoyo).
Flujo principal	1. El Director(a) ingresa al sistema y busca el expediente a concluir. 2. Verifica que las áreas de Trabajo Social, Enfermería y Psicología hayan completado sus notas finales. 3. Selecciona la opción "Generar Carta de Cierre". 4. El sistema extrae automáticamente las fechas de ingreso/egreso y el motivo de salida. 5. El actor aprueba y sella digitalmente (o imprime) la carta. 6. El sistema cambia el estado del expediente a "Cerrado".
Flujos alternos	Falta de notas finales: Si alguna de las áreas interdisciplinarias no ha cerrado su intervención en el sistema, el botón de generar carta aparece bloqueado hasta que se cumpla este requisito institucional.

Postcondiciones	El expediente queda bloqueado para nuevas ediciones y pasa a resguardo institucional bajo principios de confidencialidad para fines estadísticos y de auditoría.
------------------------	--

Nombre	Gestión de Inventario de Cocina (Alimentos)
Objetivo	Controlar las entradas, salidas y existencias físicas de los insumos alimenticios utilizados en el albergue para garantizar la alimentación de los usuarios.
Actor principal	Encargado de Cocina, Encargado de Turno.
Precondiciones	El usuario debe haber iniciado sesión en el sistema con un rol que tenga los permisos adecuados para modificar el módulo de cocina.
Flujo principal	1. El actor ingresa al módulo de "Inventario de Cocina". 2. El sistema despliega la lista actual de insumos (perecederos y no perecederos) con sus cantidades y fechas de caducidad. 3. El actor selecciona la opción de "Registrar Movimiento" (Entrada por donación/compra o Salida por consumo diario). 4. Ingresar el artículo, la cantidad y una breve justificación o vale asociado.

	5. Guarda el registro y el sistema actualiza el total de existencias en la base de datos.
Flujos alternos	Insumo próximo a caducar o por agotarse: Al visualizar la lista o registrar una salida, el sistema resalta en rojo los insumos que están por debajo del stock mínimo de seguridad o cerca de su fecha de caducidad para emitir una alerta visual.
Postcondiciones	El stock de alimentos queda actualizado y se genera un registro histórico del movimiento (quién lo hizo, cuándo y qué cantidad) para futuras auditorías.

Diagrama de Casos de uso



Reglas de negocio

1. Ingreso voluntario y condicionado:

Toda persona usuaria deberá aceptar voluntariamente su ingreso y permanencia

en el albergue, cumpliendo con el Reglamento Interno y el Código de Conducta institucional.

2. Validación de criterios de elegibilidad:

Antes de autorizar el ingreso, el personal responsable deberá realizar una entrevista inicial, detectar la situación de vulnerabilidad y verificar que la persona cumpla con los criterios establecidos para recibir atención.

3. Registro obligatorio de expediente:

Todo ingreso autorizado deberá generar la apertura de un expediente o Informe Clínico Unificado, en el cual se registren los datos generales, motivo de ingreso, antecedentes e intervenciones realizadas.

4. Atención interdisciplinaria:

La atención de cada usuario deberá involucrar, según corresponda, las áreas de Trabajo Social, Enfermería y Psicología, garantizando una valoración integral durante las primeras 72 horas.

5. Elaboración del Plan Individual de Atención:

Durante el proceso de diagnóstico integral, se deberá elaborar un Plan Individual de Atención que defina objetivos, acciones por área y seguimiento correspondiente.

6. Seguimiento mensual obligatorio:

Cada usuario deberá contar con seguimiento interdisciplinario mensual, registrando avances, dificultades, acuerdos y recomendaciones.

7. Evaluación a los tres meses:

Al cumplirse tres meses de estancia, se deberá realizar una evaluación interdisciplinaria para determinar si el usuario continúa hasta un máximo de seis meses, egresa o es canalizado a otra institución.

8. Límite de permanencia:

La estancia mínima será de 5 días, la estancia media de 3 meses y la estancia máxima de 6 meses, salvo evaluación y justificación interdisciplinaria.

9. Egreso documentado:

Todo egreso deberá quedar registrado mediante informe final interdisciplinario, carta o acta de egreso y cierre formal del expediente.

10. Confidencialidad y protección de datos:

La información de las personas usuarias deberá manejarse bajo principios de confidencialidad, resguardo institucional y protección de datos personales.

11. Canalización externa cuando sea necesario:

Cuando la condición social, psicológica o de salud del usuario exceda la capacidad institucional, se deberá realizar la canalización correspondiente a instancias competentes.

12. Atención con enfoque de derechos humanos:

Todas las acciones del sistema deberán alinearse con los principios de dignidad humana, no discriminación, inclusión social y atención integral.

13. Descripción del proyecto.

Nombre del proyecto: Ajo-olnáj (Tu Techo)

Nombre del cliente o usuario objetivo: Secretaría de Inclusión.

Integrantes del equipo:

- a. Manuel David Castro Blanco
- b. Pablo Leonel Salomon Campos
- c. Bryant Maximiliano Dzúl Pérez
- d. Gerardo Alexander Díaz León

Objetivo general del sistema: Es la regulación del ingreso, atención primaria, diagnóstico integral, seguimiento mensual, evaluación a tres meses y egreso de las personas usuarias.

Problema que busca resolver: Realizar un sistema para las personas en situación de calle, abandono social, exclusión, movilidad humana o carencia de redes de apoyo.

14. Metodología seleccionada

Indicar la metodología que utilizará el equipo para desarrollar su proyecto: Cascada

+ Kanban

Justificación.

¿Por qué seleccionaron esta metodología? Porque en este proyecto necesita una planificación estructurada pero también requiere flexibilidad para gestionar las tareas diarias

¿Qué ventajas ofrece para su proyecto?

Nos ofrece una planificación clara y específica, ya que con la metodología de Cascada definimos desde el principio los requisitos del proyecto, diseño del sistema, desarrollo, pruebas, etc.

Asimismo, implementamos el método Kanban, utilizando un tablero visual en Notion que nos permite monitorear las tareas pendientes, en progreso y completadas.

Existe una mejor organización gracias a Kanban ya que nos permite priorizar las actividades dentro de cada fase

¿Qué características del sistema hacen que sea adecuada?

El proyecto cuenta con objetivos claramente definidos, lo que permite una comprensión precisa del problema que el sistema pretende resolver. Se implementa un enfoque de desarrollo por etapas, permitiendo la división del proyecto en fases distintas, tales como análisis de requerimientos, diseño, desarrollo y pruebas, entre otras. Las tareas se identifican y se subdividen en actividades específicas y manejables, facilitando el seguimiento y la gestión mediante el uso de un tablero de tareas. A pesar del tiempo de desarrollo limitado, se ha establecido un cronograma detallado, garantizando el cumplimiento de los plazos establecidos.

¿Qué beneficios esperan obtener?

1. Mejorar organización del proyecto
2. Mayor control del avance
3. Mejor distribución del trabajo
4. Documentación completa
5. Reducción de errores
6. Mayor comunicación y coordinación

15. Organización del equipo

1. Definir los roles que existirán dentro del proyecto:
2. Nombre:
3. Rol asignado
4. Responsabilidades principales

Integrante	Rol	Responsabilidades
Manuel David Castro Blanco	Product Owner / Analista Database Administrator Backend Developer	Define los requisitos y necesidades del sistema, organiza funcionalidades y sirve como enlace entre el cliente y el equipo de desarrollo.

	Documentación / Manuales	
--	-----------------------------	--

		Diseña, administra y optimiza la base de datos, asegurando el correcto almacenamiento, seguridad e integridad de la información. Desarrolla la lógica interna del sistema, APIs, autenticación y comunicación con la base de datos. Elabora manuales técnicos y de usuario, documentando el funcionamiento, instalación y uso correcto del sistema.
--	--	--

Bryan Maximiliano Dzul Perez	Product Owner / Analista Backend Developer	Define los requisitos y necesidades del sistema, organiza funcionalidades y sirve como enlace entre el cliente y el equipo de desarrollo. Desarrolla la lógica interna del sistema, manejo de servidores, APIs, autenticación y comunicación con la base de datos.
---	--	--

Pablo Leonel Salomon Campos	UI/UX Designer Frontend Developer	Diseña la apariencia y experiencia del sistema, buscando que sea visualmente atractivo, intuitivo y fácil de usar. Desarrolla la parte visual e interactiva del sistema que utiliza el usuario, conectando el diseño con las funcionalidades.
--	---	---

Gerardo Alexander Díaz León	QA / Testing Frontend Developer	Realiza pruebas al sistema para detectar errores, validar funcionalidades y asegurar la calidad del software. Desarrolla la parte visual e interactiva del sistema que utiliza el usuario, conectando el diseño con las funcionalidades.
--------------------------------	--	---

16. Roles y responsabilidades

UI/UX Designer (Salomon Campos) Diseña la apariencia y experiencia del sistema, buscando que sea visualmente atractivo, intuitivo y fácil de usar.

Product Owner / Analista (David Castro, Maximiliano Dzul) Define los requisitos y necesidades del sistema, organiza funcionalidades y sirve como enlace entre el cliente y el equipo de desarrollo.

QA / Testing (Alexander Diaz) Realiza pruebas al sistema para detectar errores, validar funcionalidades y asegurar la calidad del software.

Database Administrator (David Castro) Diseña, administra y optimiza la base de datos, asegurando el correcto almacenamiento, seguridad e integridad de la información.

Frontend Developer (Salomon Campos, Alexander Diaz) Desarrolla la parte visual e interactiva del sistema que utiliza el usuario, conectando el diseño con las funcionalidades.

Backend Developer (David Castro, Maximiliano Dzul) Desarrolla la lógica interna del sistema, manejo de servidores, APIs, autenticación y comunicación con la base de datos.

Documentación / Manuales (David Castro) Elabora manuales técnicos y de usuario, documentando el funcionamiento, instalación y uso correcto del sistema.

17. Planificación del proyecto Organización del trabajo

Para el desarrollo del sistema se utilizará una combinación de las metodologías Cascada y Kanban. Cascada permitirá seguir un proceso ordenado por etapas, mientras que Kanban facilitará el control y seguimiento de las actividades mediante un tablero visual.

Asignación de tareas

Las tareas serán definidas por el Product Owner y distribuidas entre los miembros del equipo según sus responsabilidades y habilidades. Cada integrante conocerá

claramente las actividades que debe realizar y los tiempos establecidos para su cumplimiento.

Seguimiento del avance

El progreso del proyecto se controlará mediante un tablero de tareas de Notion dividido en columnas como "Pendiente", "En Proceso" y "Completado". Esto permitirá visualizar el estado de cada actividad, identificar retrasos y verificar el cumplimiento de los objetivos establecidos en cada fase.

Gestión de cambios o incidencias

Cualquier cambio solicitado o problema detectado durante el desarrollo será registrado y analizado por el equipo. El Product Owner evaluará el impacto del cambio y coordinará las acciones necesarias para implementarlo o corregir la incidencia sin afectar significativamente el cronograma del proyecto.

Comunicación del equipo

La comunicación se realizará mediante reuniones de videollamadas y por medio de mensajería. Estas reuniones permitirán informar avances, resolver dudas, coordinar actividades y mantener a todos los integrantes actualizados sobre el estado del proyecto.

Organización del trabajo

Las actividades del proyecto se gestionan siguiendo las fases definidas por la metodología Cascada y utilizando un tablero Kanban (notion) para visualizar y controlar el avance de las tareas. Esto permitirá mantener una organización clara y un seguimiento constante durante todo el desarrollo del sistema.

Asignación de tareas

Las tareas serán asignadas por el Líder del Proyecto de acuerdo con las responsabilidades de cada integrante del equipo. Los desarrolladores se encargaron de

la programación e implementación de funcionalidades, mientras que el responsable de calidad realizará las pruebas y validaciones correspondientes. Cada tarea tendrá un responsable y una fecha estimada de finalización.

Seguimiento del avance

El avance del proyecto será monitoreado mediante reuniones periódicas y un tablero Kanban dividido en las categorías "Pendiente", "En Proceso" y "Completado". Esto permitirá conocer el estado de cada actividad, detectar retrasos y verificar el cumplimiento de los objetivos establecidos para cada fase del proyecto.

Comunicación del equipo

La comunicación entre los integrantes se realizará mediante reuniones de seguimiento y herramientas de mensajería digital como WhatsApp y Meet. Estas reuniones permitirán informar avances, resolver dudas, coordinar actividades y garantizar que todos los miembros del equipo estén alineados con los objetivos del proyecto.

De esta manera se busca mantener una organización eficiente, una comunicación constante y un adecuado control del desarrollo del sistema para cumplir con los tiempos y objetivos establecidos.

18. Etapas o iteraciones

Debido a que la metodología seleccionada es Cascada, el proyecto se desarrollará mediante fases secuenciales. Cada etapa deberá completarse antes de avanzar a la siguiente.

Etapa 1: Análisis de Requerimientos

Objetivo:

Identificar y documentar las necesidades del sistema de registro.

Actividades:

- a. Reunión con el cliente para recopilar requisitos.
- b. Identificación de funcionalidades y restricciones.
- c. Elaboración del documento de requisitos.

Etapa 2: Diseño del Sistema

Objetivo:

Diseñar la estructura y funcionamiento del sistema antes de iniciar la programación.

Actividades:

- d. Diseño de la base de datos.
- e. Elaboración de diagramas UML.
- f. Diseño de interfaces de usuario.

Etapa 3: Desarrollo e Implementación

Objetivo:

Construir las funcionalidades definidas durante el análisis y diseño.

Actividades:

- g. Programación del módulo de registro de beneficiarios.
- h. Desarrollo de consultas y reportes.

Etapa 4: Pruebas y Validación

Objetivo:

Verificar que el sistema funcione correctamente y cumpla con los requisitos establecidos.

Actividades:

- i. Ejecución de pruebas funcionales.
- j. Corrección de errores detectados.

k. Validación de resultados con el cliente

Etapa 5: Implementación y Entrega

Objetivo:

Poner en funcionamiento el sistema y realizar la entrega final. Actividades:

- l. Instalación del sistema.
- m. Capacitación básica a los usuarios.
- n. Entrega de documentación y versión final del software.

19. Estrategia de control de versiones

El control de versiones permitirá llevar un registro de todos los cambios realizados en el código fuente del proyecto, facilitando el trabajo colaborativo y evitando la pérdida de información.

Herramienta utilizada

Se utilizará Git como sistema de control de versiones, ya que permite registrar los cambios realizados en el proyecto, recuperar versiones anteriores y trabajar de manera colaborativa entre varios desarrolladores.

Repositorio del proyecto

El proyecto será almacenado en un repositorio de GitHub, donde se mantendrá el código fuente centralizado. Esto permitirá que todos los integrantes del equipo tengan acceso a la versión más reciente del sistema y puedan contribuir al desarrollo de forma organizada.

Estrategias de ramas

Se utilizará una estrategia basada en ramas para separar el trabajo de cada integrante y evitar conflictos en el código:

- main: Contendrá la versión estable y funcional del proyecto.
- DEV: Servirá para integrar las nuevas funcionalidades antes de pasarlas a producción.

- David: Rama donde trabajará solamente Manuel David Castro Blanco.
- Max: Rama donde trabajará solamente Bryan Maximiliano Dzul Perez.
- Salomon: Rama donde trabajará solamente Pablo Leonel Salomon Campos
- Alexander: Rama donde trabajará solamente Gerardo Alexander Díaz León

De esta forma, cada cambio podrá ser probado antes de incorporarse a la versión principal del sistema.

Procedimiento para integración de cambios

Cuando un desarrollador termine una tarea, enviará sus cambios a su rama correspondiente mediante un commit. Posteriormente, realizará una solicitud de integración (Pull Request) hacia la rama sus ramas correspondientes. El Líder del Proyecto revisará los cambios y verificará que funcionen correctamente antes de aprobar su integración. Una vez validados y probados, los cambios se incorporarán a la rama main, manteniendo así la estabilidad y calidad del proyecto.

Esta estrategia permitirá mantener un desarrollo ordenado, reducir errores y facilitar la colaboración entre los miembros del equipo.

20. Gestión de riesgo

La gestión de riesgos consiste en identificar posibles situaciones que podrían afectar el desarrollo del proyecto y establecer acciones preventivas para reducir su impacto. Esto permite anticipar problemas y tomar medidas oportunas para asegurar el cumplimiento de los objetivos y tiempos establecidos.

Riesgo	Impacto	Acción Preventiva
--------	---------	-------------------

Retrasos en las entregas	Alto	Realizar seguimiento semanal de las actividades y verificar el cumplimiento de los plazos establecidos.
Cambios en los requerimientos	Medio	Mantener una comunicación constante con el cliente y validar periódicamente los requisitos del sistema.
Problemas técnicos durante el desarrollo	Alto	Realizar investigaciones, pruebas tempranas y prototipos antes de implementar funcionalidades complejas.
Falta de comunicación entre los integrantes	Medio	Programar reuniones periódicas y utilizar herramientas de comunicación para compartir avances y resolver dudas.
Errores en el software detectados en etapas finales	Alto	Aplicar pruebas continuas durante el desarrollo y revisiones de calidad en cada fase del proyecto.
Ausencia de algún integrante del equipo	Medio	Documentar el trabajo realizado y mantener actualizada la información para que otros miembros puedan continuar las actividades si es necesario.

¿Qué es nuestra Definition of Done?

La **Definition of Done** es la lista de criterios mínimos que **toda funcionalidad** del sistema Ajooinaj debe cumplir antes de considerarse *terminada* e integrarse a la rama principal. No basta con que “funcione en mi máquina”: una funcionalidad está *Done* solo cuando cumple **todos** los criterios de esta lista.

Estos criterios están adaptados a nuestro contexto real: un proyecto Laravel 12 con PostgreSQL, arquitectura modular por dominios, RBAC propio y trabajo en equipo mediante Git y *Pull Requests*.

Criterios (una funcionalidad está Done cuando...)

1. **Cumple los requisitos definidos.** La funcionalidad hace exactamente lo que pide el requerimiento funcional acordado con el equipo/cliente, sin faltantes.
2. **Sigue nuestras 5 reglas de Clean Code.** Nombres del dominio, lógica en servicios, constantes con nombre, validación en FormRequests y comentarios del *por qué* (ver documento Reglas de Clean Code).
3. **La lógica de negocio está en un Service, no en el controlador.** El controlador permanece delgado: valida, delega en el servicio de dominio y responde.
4. **Tiene validación y autorización correctas.** Las entradas se validan y los permisos se comprueban vía FormRequest (rules() + authorize()) o Policy, respetando el RBAC y las reglas por área.
5. **Pasa las pruebas automáticas.** php artisan test se ejecuta en verde; si la funcionalidad añade lógica crítica, incluye o actualiza sus pruebas.
6. **Fue probada manualmente en el flujo real.** Se verificó en el sistema levantado (Laravel Sail) recorriendo el caso de uso completo, no solo en teoría.
7. **No presenta errores ni warnings conocidos.** Sin excepciones en logs, sin datos corruptos y sin advertencias del framework en el flujo probado.
8. **Fue revisada por otro integrante.** Existe un Pull Request revisado y aprobado por al menos un compañero distinto a quien la desarrolló.
- G. **Las migraciones aplican limpio y son no destructivas.** Si toca la base de datos, las migraciones corren sin errores y no borran datos existentes (nada de migrate:fresh/refresh/wipe sobre datos reales).
10. **Documentación actualizada e integrada a la rama principal.** El README o la documentación de la fase reflejan el cambio, el código sigue el formato PSR-12 (Laravel Pint) y quedó mergeado sin conflictos en la
11. rama principal.
12. **Notas de aplicación**
 - **Verificación de estilo (criterio 2 y 10):** el formato se normaliza con Laravel Pint antes de hacer commit, para que todo el equipo entregue el mismo estilo.
 - **Sobre las pruebas (criterio 5):** en Ajooinaj las pruebas corren sobre SQLite en memoria, pero varias migraciones usan tipos propios de PostgreSQL; por eso la lógica de base de

datos también se verifica contra PostgreSQL real antes de dar por cumplido este criterio.

- **Sobre la revisión (criterio 8):** la revisión por Pull Request es donde se confirma que se cumplen los criterios 1–7 y las reglas de Clean Code.

Compromiso del equipo

Esta Definition of Done es un acuerdo del equipo y aplica a **todas** las funcionalidades del proyecto por igual. Una tarea que no cumpla los 10 criterios **no** se marca como terminada. Su objetivo es evitar acumular deuda técnica y mantener un código organizado, mantenible y de calidad durante todo el desarrollo de Ajoonaj.

Propósito de este documento

Estas son las reglas de **Clean Code** que el equipo se compromete a seguir durante **todo** el desarrollo de Ajoonaj. No son reglas copiadas de internet: se eligieron a partir del estado actual de nuestro sistema (Laravel 12 + PHP 8.2 + PostgreSQL, con arquitectura modular por dominios en `app/Domain/*`) y de las decisiones técnicas que ya tomamos en las Fases 1–7. El objetivo es mantener un código **legible, mantenible y consistente** para todo el equipo.

Regla 1 — Nombres descriptivos y en el lenguaje del dominio

La regla. Clases, métodos, variables y rutas se nombran de forma descriptiva y usando el vocabulario real del albergue (español). Un nombre debe decir *qué* es o *qué* hace, sin abreviaturas ambiguas.

- Clases de servicio con verbo + sustantivo: RegistrarIngresoService, AbrirExpedienteService, RegistrarEgresoService.
- Método público de acción del servicio: ejecutar().
- Entidades con el nombre del negocio: PersonaUsuaría, Expediente, Estancia, PeriodoEstancia, Cama.

¿Por qué la implementamos? Nuestro sistema modela un proceso real (ingreso → estancia → seguimiento clínico → egreso). Si el código habla el mismo idioma que el negocio, cualquier integrante entiende qué hace una clase sin leer su implementación.

¿Qué problema evita? La ambigüedad y la “traducción mental” entre el negocio y el código (`$x`, `$data`, `proc()`), que provoca errores y hace lento el mantenimiento.

¿Cómo la aplicamos en el proyecto? Todo servicio de dominio se nombra `<Acción><Entidad>Service` y expone `ejecutar()`. Los modelos usan el nombre del dominio. Las tablas, columnas y roles ya siguen esta convención (`persona_usuario`, `numero_episodio`, `roles_trabajo_social` / `enfermeria` / `psicologia`).

Regla 2 — La lógica de negocio vive en Servicios de dominio, no en los controladores

La regla. Los controladores son **delgados**: reciben la petición, delegan el trabajo a un Servicio de dominio (`app/Domain/*/Services`) y devuelven la respuesta. Ninguna regla de negocio se escribe dentro de un controlador.

¿Por qué la implementamos? La lógica del albergue (cómo se calcula un reingreso, cómo se asigna una cama, cómo se cierra una estancia) es el corazón del sistema y debe poder reutilizarse y probarse de forma aislada.

¿Qué problema evita? Los “controladores Dios” que hacen de todo y la duplicación de lógica entre pantallas. También evita que una misma regla quede escrita de dos maneras distintas.

¿Cómo la aplicamos en el proyecto? RegistrarIngresoService::ejecutar() orquesta — dentro de una transacción— la creación de la Estancia, su PeriodoEstancia inicial y la asignación de Cama, calculando el numero_episodio del reingreso. El controlador solo valida (vía FormRequest) y llama al servicio.

Regla 3 — Nada de números ni cadenas “mágicas”: usar constantes con nombre

La regla. Ningún valor de negocio se escribe “a mano” repartido por el código. Se declara **una sola vez** como constante con nombre en el modelo correspondiente.

¿Por qué la implementamos? Reglas como “la permanencia inicial es de 3 meses y el tope es de 6” son decisiones del negocio que pueden cambiar; deben vivir en un único lugar.

¿Qué problema evita? Valores repetidos e inconsistentes (un 3 aquí, un “Usuario” allá) que, al cambiar, se olvidan en algún archivo y generan bugs difíciles de rastrear.

¿Cómo la aplicamos en el proyecto? Constantes reales ya en el código:

```
// app/Domain/Estancias/Models/Estancia.php public
const ESTATUS_USUARIO      = 'Usuario'; public
const ESTATUS_RESIDENTE    = 'Residente'; public
const MESES_PERIODO_INICIAL = 3;
public const MESES_TOPE_PERMANENCIA = 6;
```

Los servicios usan Estancia::MESES_PERIODO_INICIAL en lugar del número 3.

Regla 4 — Validación y autorización fuera del controlador (FormRequest / Policy)

La regla. Las reglas de “qué datos son válidos” y “quién tiene permiso” viven en un FormRequest (métodos rules() y authorize()) o en una Policy, nunca sueltas dentro del controlador.

¿Por qué la implementamos? Ajoonaj tiene un RBAC propio con 7 roles y reglas de autorización por área clínica (Psicología no puede escribir el diagnóstico médico, etc.). Esa lógica debe estar centralizada y ser reutilizable.

¿Qué problema evita? Reglas de validación/seguridad dispersas y duplicadas, y controladores inseguros donde es fácil olvidar una comprobación de permisos.

¿Cómo la aplicamos en el proyecto? StoreDiagnosticoRequest define rules() (el área es obligatoria y válida, la descripción no excede 5000 caracteres) y authorize() (el usuario puede editar esa área, con cortocircuito para admin). El controlador recibe la petición ya validada y autorizada.

```
// app/Http/Requests/Clinico/StoreDiagnosticoRequest.php
public function authorize(): bool
{
    $user = $this->user();
    return $user->hasRole('admin')
```

```

    || app(DiagnosticoPolicy::class)->puedeEditarArea($user, (string) $this-
>input('area'));
}

```

Regla 5 — Comentarios que explican el por qué, y PHPDoc con tipos

La regla. El código explica el *qué*; los comentarios se reservan para el *por qué* (la intención o la regla de negocio). Los servicios llevan PHPDoc con los tipos de sus parámetros y su retorno. No se escriben comentarios obvios.

¿Por qué la implementamos? Hay decisiones de negocio que no son evidentes leyendo el código (p. ej. que un reingreso reutiliza el expediente pero incrementa el episodio). Documentar la intención evita que alguien “corrija” algo que en realidad es correcto.

¿Qué problema evita? Los comentarios obvios (`// crea la estancia`) que envejecen, se desincronizan del código y terminan mintiendo.

¿Cómo la aplicamos en el proyecto? Cada servicio abre con un docblock que resume su responsabilidad y la regla de negocio, y comentarios puntuales explican decisiones no evidentes:

```

// Reingreso: numero_episodio = max(previos del expediente) + 1.
$ultimoEpisodio = Estancia::where('expediente_id', $expediente->id)-
>max('numero_episodio') ?? 0;

```

Convenciones de estilo complementarias

Además de las 5 reglas principales, el equipo acuerda estos lineamientos de apoyo, alineados con la lista de aspectos sugerida en la actividad:

Aspecto	Convención acordada
Nombres de variables	camelCase para variables/propiedades, en español y descriptivas (\$fechaEgresoPrevista, \$ultimoEpisodio).
Nombres de funciones/métodos	Verbo en infinitivo + sustantivo (ejecutar, puedeEditarArea, asignarCama). Clases en PascalCase.
Organización de carpetas	Modular por dominio: app/Domain/<Dominio>/{Models,Services,Policies}. Nada de “carpetas cajón”.
Longitud de funciones	Funciones cortas y de responsabilidad única; si un método crece, se extrae a otro servicio/método privado.
Uso de comentarios	Solo para el por qué (ver Regla 5). Prohibido comentar lo obvio o dejar código muerto comentado.
Manejo de constantes	Valores de negocio como constantes con nombre en el modelo (ver Regla 3). Nada de “magic numbers/strings”.

Aspecto	Convención acordada
Formato y estilo del código	Estándar PSR-12, aplicado automáticamente con Laravel Pint antes de cada commit (formato uniforme para todo el equipo).

Compromiso del equipo

Estas reglas son de cumplimiento obligatorio y forman parte de nuestra **Definition of Done**: ningún cambio se considera terminado si no las respeta. Su cumplimiento se verifica en la revisión por Pull Request antes de integrar a la rama principal.

Introducción

El presente documento establece el **Plan de Pruebas** del Sistema de Gestión Institucional del Albergue "Ajo'olnáj — Tu Techo". Su propósito es definir la estrategia, los escenarios y los criterios que se emplearán para verificar que el software cumple correctamente con los requerimientos funcionales establecidos en la especificación de requerimientos (SRS) organizada por fases.

La planificación de pruebas se realiza antes de la finalización del desarrollo completo del sistema, siguiendo las buenas prácticas de Ingeniería de Software, con el fin de anticipar hallazgos, reducir costos de corrección y garantizar la calidad del producto entregado.

Descripción del Proyecto Nombre del proyecto

Sistema de Gestión Institucional — Albergue "Ajo'olnáj — Tu Techo" Objetivo general

Desarrollar una plataforma digital que integre la gestión operativa, clínica y administrativa del albergue, sustituyendo los procesos realizados en formato papel (bitácoras, formatos clínicos, inventarios y listas de verificación) por un sistema web centralizado, seguro y basado en roles.

Problema que resuelve

El albergue atiende a personas en situación de calle en la ciudad de Campeche, México. Actualmente, los procesos de admisión, seguimiento clínico, control de estancias, inventario de medicamentos e insumos, y reportes se manejan mediante documentos de Word, Excel y formato impreso. Esto provoca:

- Pérdida o duplicación de información.
- Dificultad para dar seguimiento integral a cada residente.

- Falta de trazabilidad en movimientos de inventario.
- Ausencia de alertas automáticas para vencimiento de permanencia o medicamentos.
- Limitación en la generación de reportes para toma de decisiones.

Tecnologías utilizada

Componente	Tecnología	Versión
Backend	PHP / Laravel	8.2.29 / 12.x
Frontend	Blade + Tailwind CSS + Alpine.js	—
Base de datos	PostgreSQL	12.17
Autenticación	Laravel Breeze	—
Assets	Vite	—
Contenedores	Docker / Laravel Sail	—
Node.js	Node.js / npm	22.17.0 / 10.9.2

Arquitectura general del sistema

El sistema utiliza una arquitectura modular basada en **Domain-Driven Design** con 11 dominios bajo **app/Domain/**. Cada dominio contiene sus propios Modelos, Servicios y Políticas de autorización. Las rutas se organizan en archivos modulares bajo **routes/modules/**. El sistema implementa un **RBAC personalizado** con 7 roles y permisos.

Módulos principales

#	Módulo	Descripción
1	Acceso y RBAC	Autenticación, 7 roles, permisos granulares, gestión de empleados y usuarios
2	Catálogos	Datos maestros (20+ catálogos), cascade geográfico, catálogos simples genéricos

3	Expedientes y Admisión	Registro de personas, expedientes con folio automático, búsqueda difusa (trigram)
---	------------------------	---

4	Estancias y Camas	Gestión de estancias, periodos de permanencia (3-6 meses), asignación de camas, pertenencias
5	Bitácora	Registro diario de atención, servicios, visitas e incidencias
6	Clínico	Cédula clínica interdisciplinaria, plan individual de atención, signos vitales
7	Valoraciones	Valoraciones por área (Trabajo Social, Enfermería, Psicología)
8	Egresos y Evaluaciones	Evaluaciones trimestrales, egreso, canalizaciones, cierre de expediente
9	Inventario	Medicamentos (lotes, caducidad, administración), insumos, recepciones, requisiciones

10	Operación	Listas de verificación (checklists) e inspecciones por turno
11	Sistema	Adjuntos, auditoría, notificaciones

Estrategia General de Pruebas

Tipos de pruebas

El equipo realizará los siguientes tipos de prueba:

Tipo de prueba	Descripción	Alcance
Unitarias	Verifican métodos y funciones individuales de modelos, servicios y políticas	Lógica de negocio aislada
De integración	Verifican la comunicación correcta entre módulos (controlador-servicio-modelo, rutas-vistas)	Flujo entre dos o más componentes
End-to-End (E2E)	Simulan recorridos completos del usuario desde la interfaz	Procesos de negocio completos
De regresión	Se ejecutan después de cada cambio significativo para confirmar que funcionalidades existentes no se afectaron	Todo el sistema
De validación de datos	Verifican el manejo correcto de entradas inválidas, vacías y límites	Formularios y endpoints

Momento de ejecución

Tipo de prueba	Fase de desarrollo
Unitarias	Continua, durante el desarrollo de cada módulo (cada fase)

De integración	Al finalizar cada fase, al conectar módulos entre sí
----------------	--

E2E	Después de completar las fases 1-7 (módulos core operativos)
De regresión	Después de cada cambio o corrección de errores
De validación de datos	Durante el desarrollo de cada formulario y endpoint

Responsables

Tipo de prueba	Responsable
Unitarias	El desarrollador que implementó el componente
De integración	Un integrante del equipo diferente al desarrollador
E2E	Cualquier integrante del equipo (simulando el rol de usuario)
De regresión	El desarrollador que realizó el cambio
De validación de datos	El desarrollador del módulo + revisión cruzada

Herramientas para documentar y registrar resultados

Herramienta	Uso
PHPUnit / Pest (incluido en Laravel)	Ejecución automatizada de pruebas unitarias y de integración
Markdown en el repositorio	Registro manual de escenarios E2E y resultados observados
GitHub Issues	Registro de errores encontrados durante las pruebas
Bitácora de auditoría del sistema	Verificación de trazabilidad post-cambio

Navegador web (Chrome/Firefox)	Ejecución manual de pruebas E2E
Consola del navegador	Verificación de errores de JavaScript y peticiones AJAX

Justificación

La estrategia se enfoca en pruebas unitarias como primera línea de defensa, dado que el sistema maneja lógica de negocio compleja (cálculo de permanencia, control de stock por lotes, RBAC por roles). Las pruebas de integración son críticas porque los 11 dominios interactúan frecuentemente. Las E2E validan los flujos reales que realizarán los usuarios del albergue (personal de trabajo social, enfermería, psicología, cocina y dirección).

Plan de Pruebas Unitarias

PU-01: Cálculo de fecha de egreso prevista

Campo	Descripción
Identificador	PU-01
Módulo	Estancias
Función o método a probar	Estancia::calcularFechaEgresoPrevista() o lógica equivalente en el servicio de estancias
Objetivo de la prueba	Verificar que al crear una estancia, la fecha de egreso prevista se calcule correctamente sumando 3 meses a la fecha de ingreso
Datos de entrada	Fecha de ingreso: 2026-01-15
Resultado esperado	Fecha de egreso prevista: 2026-04-15 (exactamente 3 meses después)
Resultado obtenido	Pendiente
Estado	Pendiente

PU-02: Generación automática de folio de expediente

Campo	Descripción
Identificador	PU-02
Módulo	Expedientes

Función o método a probar	Expediente::generarFolio() o servicio de expedientes
----------------------------------	---

Objetivo de la prueba	Verificar que el folio se genere con el formato correcto: VIS-AAAA-NNNNN para visitantes o RES-AAAA-NNNNN para residentes, con secuencia incremental
Datos de entrada	Tipo de persona: "Residente", Año actual: 2026, Último folio existente: RES-2026-00003
Resultado esperado	Nuevo folio generado: RES-2026-00004
Resultado obtenido	Pendiente
Estado	Pendiente

PU-03: Verificación de permisos por rol (RBAC)

Campo	Descripción
Identificador	PU-03
Módulo	Acceso
Función o método a probar	Role y Permission models — Gates/Policiés del sistema RBAC
Objetivo de la prueba	Verificar que un usuario con rol direccion (solo lectura) no pueda ejecutar acciones de escritura (crear, editar, eliminar) en ningún módulo
Datos de entrada	Usuario con rol direccion , intento de acceder a POST /empleados
Resultado esperado	Respuesta HTTP 403 (Forbidden) — acceso denegado

Resultado obtenido	Pendiente
Estado	Pendiente

PU-04: Cálculo de stock disponible de medicamento por lotes

Campo	Descripción
--------------	--------------------

Identificador	PU-04
Módulo	Inventario
Función o método a probar	Medicamento::stockDisponible() o servicio de medicamentos — suma de cantidades de todos los lotes activos con stock > 0
Objetivo de la prueba	Verificar que el stock total de un medicamento se calcule sumando las cantidades disponibles de todos sus lotes (considerando movimientos de entrada y salida)
Datos de entrada	Medicamento "Paracetamol 500mg": Lote A (100 unidades), Lote B (50 unidades), movimiento de salida de 30 unidades del Lote A
Resultado esperado	Stock disponible: 120 unidades (70 del Lote A + 50 del Lote B)
Resultado obtenido	Pendiente
Estado	Pendiente

PU-05: Validación de límite de permanencia (6 meses máximo)

Campo	Descripción
--------------	--------------------

Identificador	PU-05
Módulo	Estancias
Función o método a probar	Servicio de estancias — validación al intentar extender una estancia
Objetivo de la prueba	Verificar que el sistema impida extender una estancia cuando ya se ha alcanzado el límite máximo de 6 meses de permanencia

Datos de entrada	Estancia con fecha de ingreso 2026-01-01 , fecha de egreso prevista actual 2026-06-30 (6 meses), intento de extensión
Resultado esperado	El sistema rechaza la extensión y muestra mensaje: "La estancia ha alcanzado el límite máximo de permanencia (6 meses)"
Resultado obtenido	Pendiente
Estado	Pendiente

PU-06: Validación de administración de medicamento con stock insuficiente

Campo	Descripción
Identificador	PU-06
Módulo	Inventario
Función o método a probar	Servicio de administración de medicamentos

Objetivo de la prueba	Verificar que el sistema impida administrar un medicamento cuando el stock disponible es menor a la cantidad solicitada
Datos de entrada	Medicamento con stock disponible: 5 unidades, cantidad a administrar: 10 unidades
Resultado esperado	El sistema rechaza la administración y retorna error de stock insuficiente
Resultado obtenido	Pendiente
Estado	Pendiente

PU-07: Búsqueda difusa de personas por trigram (prevención de duplicados)

Campo	Descripción
--------------	--------------------

Identificador	PU-07
Módulo	Expedientes
Función o método a probar	Endpoint GET /expedientes/api/buscar-personas con parámetro de búsqueda
Objetivo de la prueba	Verificar que la búsqueda difusa con trigram encuentre personas con nombres similares (tolerancia a errores de escritura)
Datos de entrada	Nombre registrado: "María Elena López", término de búsqueda: "Maria Lopez"

Resultado esperado	La búsqueda retorna el registro de "María Elena López" como resultado coincidente
Resultado obtenido	Pendiente
Estado	Pendiente

Plan de Pruebas de Integración

PI-01: Inicio de sesión → Acceso al Dashboard

Campo	Descripción
Identificador	PI-01
Módulos involucrados	Autenticación (Breeze) → RBAC → Dashboard
Objetivo de la prueba	Verificar que un usuario autenticado con credenciales correctas acceda al dashboard y que el contenido del panel se adapte a su rol asignado
Flujo esperado	1. El usuario ingresa email y contraseña en /login . 2. Laravel Breeze valida las credenciales. 3. El sistema verifica que la cuenta esté activa (activo = true). 4. Se redirige

	a /dashboard . 5. El dashboard muestra menú y módulos según el rol del usuario
--	---

Resultado esperado	El usuario accede al dashboard y visualiza únicamente los módulos correspondientes a su rol. Un administrador ve todos los módulos; un trabajador social ve expedientes, estancias y egresos
Estado	Pendiente

Justificación: Esta integración es crítica porque es el punto de entrada principal al sistema. Un fallo aquí bloquea completamente el acceso. Además, la personalización del menú según el rol valida que el RBAC funcione correctamente desde el inicio.

PI-02: Registro de empleado → Creación de usuario → Asignación de rol

Campo	Descripción
Identificador	PI-02
Módulos involucrados	Empleados (Acceso) → Usuarios (Breeze) → Roles (RBAC)
Objetivo de la prueba	Verificar que al crear un empleado, se genere su cuenta de usuario, se le asigne el rol correcto y pueda iniciar sesión con las credenciales generadas
Flujo esperado	1. Admin crea un empleado con datos personales y área. 2. El sistema genera la cuenta de usuario (email, contraseña temporal). 3. Se asigna el rol correspondiente. 4. El nuevo usuario inicia sesión. 5. El sistema carga los permisos del rol

	asignado
Resultado esperado	El empleado queda registrado, su cuenta de usuario está activa, el rol está asignado correctamente y puede acceder a los módulos que le corresponden
Estado	Pendiente

Justificación: La creación de empleados es una operación transaccional compleja que involucra empleado + documentos obligatorios + usuario + rol. Si falla esta cadena, el sistema no puede incorporar nuevo personal operativo.

PI-03: Apertura de expediente → Creación de estancia → Asignación de cama

Campo	Descripción
Identificador	PI-03
Módulos involucrados	Personas → Expedientes → Estancias → Camas
Objetivo de la prueba	Verificar el flujo completo de admisión: desde el registro de una persona hasta la asignación de una cama en un dormitorio

Flujo esperado	1. Se registra una persona usuaria (con datos personales). 2. Se crea su expediente (folio automático). 3. Se abre una estancia asociada al expediente. 4. Se asigna una cama disponible en un dormitorio. 5. La cama aparece como ocupada en el tablero de camas
-----------------------	--

Resultado esperado	La persona queda registrada con su expediente, estancia activa y cama asignada. El tablero de camas refleja la ocupación correctamente
Estado	Pendiente

Justificación: Este es el flujo principal del albergue: admitir una persona y ubicarla. La integración entre estos cuatro módulos es esencial para la operación diaria. Un fallo aquí impediría el ingreso de nuevos residentes.

PI-04: Estancia activa → Cédula clínica → Valoraciones → Egreso

Campo	Descripción
Identificador	PI-04
Módulos involucrados	Estancias → Clínico → Valoraciones → Egresos
Objetivo de la prueba	Verificar que el ciclo de vida clínico de una estancia funcione correctamente: desde la evaluación interdisciplinaria inicial hasta el egreso

Flujo esperado	1. Con una estancia activa, Trabajo Social realiza la cédula clínica. 2. Enfermería registra signos vitales. 3. Psicología registra su valoración. 4. Se autoriza el plan individual. 5. Se realiza la evaluación trimestral (requiere los 3 dictámenes). 6. Trabajo Social genera el acta de egreso. 7. Se cierra el expediente
Resultado esperado	Todos los registros clínicos quedan asociados a la estancia, la evaluación trimestral se completa con los 3 dictámenes, el egreso se registra correctamente y el expediente se marca como cerrado

Estado	Pendiente
---------------	-----------

Justificación: Este flujo representa el ciclo de vida completo de un residente en el albergue. La integración entre módulos clínicos y de egreso es vital para garantizar la continuidad de la atención y el cumplimiento de los protocolos institucionales.

PI-05: Inventario de medicamentos → Administración a residente → Bitácora

Campo	Descripción
Identificador	PI-05
Módulos involucrados	Inventario (Medicamentos) → Estancias → Bitácora
Objetivo de la prueba	Verificar que al administrar un medicamento a un residente, se descuenta del stock y quede registrado en la bitácora de atención del día

Flujo esperado	1. Enfermería selecciona un medicamento con stock disponible. 2. Registra la administración a un residente específico. 3. El sistema descuenta la cantidad del lote correspondiente. 4. Se crea un registro en la bitácora de atención del día
Resultado esperado	El stock del medicamento se reduce, la administración queda registrada con residente, medicamento, cantidad y fecha, y aparece en la bitácora diaria
Estado	Pendiente

Justificación: La gestión de medicamentos es un proceso crítico de seguridad. Un fallo en la integración podría provocar doble administración, stock negativo o falta de trazabilidad farmacológica.

Plan de Pruebas End-to-End (E2E)

E2E-01: Admisión completa de un nuevo residente

Campo	Descripción
Identificador	E2E-01
Nombre del escenario	Admisión completa de un nuevo residente
Actor principal	Trabajo Social
Precondiciones	El usuario tiene sesión activa con rol trabajo_social . Existen dormitorios con camas disponibles. Los catálogos de motivo de ingreso, discapacidad y adicción están poblados

Pasos a seguir	1. Iniciar sesión con credenciales de Trabajo Social. 2. Navegar a Expedientes > Nuevo Expediente. 3. Buscar la persona por nombre (verificar que no exista previamente). 4. Completar el formulario de registro: nombre, fecha de nacimiento, CURP (opcional), sexo, estado civil, escolaridad, dirección, motivo de ingreso, discapacidades, adicciones, situación de vulnerabilidad. 5. Guardar el expediente y verificar el folio generado. 6. Crear una nueva estancia asociada al expediente. 7. Navegar al tablero de camas. 8. Asignar una cama disponible en un dormitorio Transitorio. 9. Verificar que la cama aparezca como ocupada. 10. Registrar pertenencias del residente. 11. Cerrar sesión
-----------------------	---

Resultado esperado	La persona queda registrada con folio RES-2026-XXXXX , tiene una estancia activa con cama asignada, sus pertenencias quedan registradas y el tablero de camas refleja la ocupación
Estado	Pendiente

E2E-02: Proceso clínico interdisciplinario y evaluación trimestral

Campo	Descripción
Identificador	E2E-02
Nombre del escenario	Evaluación clínica interdisciplinaria y evaluación trimestral
Actor principal	Trabajo Social, Enfermería, Psicología (tres usuarios diferentes)
Precondiciones	Existe un residente con estancia activa y cama asignada. Los tres usuarios (trabajo social, enfermería, psicología) tienen credenciales válidas
Pasos a seguir	1. Trabajo Social inicia sesión y registra la cédula clínica inicial (área de Trabajo Social). 2. Trabajo Social crea el plan individual de atención. 3. Enfermería inicia sesión, registra signos vitales del residente (presión arterial, frecuencia cardíaca, temperatura, saturación de oxígeno). 4. Enfermería realiza su valoración de enfermería. 5. Psicología inicia sesión y realiza su valoración psicológica. 6. Se autoriza el plan individual (cada área autoriza su sección). 7. Trabajo Social crea la

	evaluación trimestral con los tres dictámenes. 8. Se verifica que la evaluación se complete exitosamente (requiere los 3 dictámenes)
Resultado esperado	La cédula clínica, signos vitales, valoraciones de las tres áreas y la evaluación trimestral quedan registrados y asociados a la estancia. El plan individual muestra las tres columnas correctamente. La evaluación trimestral se completa con los tres dictámenes
Estado	Pendiente

E2E-03: Egreso de residente con cierre de expediente

Campo	Descripción
Identificador	E2E-03
Nombre del escenario	Egreso de residente y cierre de expediente
Actor principal	Trabajo Social
Precondiciones	Existe un residente con estancia activa, al menos una evaluación trimestral completada y el plazo de permanencia está próximo a vencer o se requiere egreso anticipado

Pasos a seguir	1. Trabajo Social inicia sesión. 2. Navega al expediente del residente. 3. Revisa el historial de estancias y evaluaciones. 4. Registra el informe final interdisciplinario. 5. Genera el acta de egreso (solo Trabajo Social puede crear egresos). 6. Registra la causa de egreso y opcionalmente una canalización a institución externa. 7. Entrega las pertenencias pendientes del residente. 8. Cierra la estancia. 9. Verifica que el tablero de camas libere la cama. 10. Cierra el expediente. 11. Verifica que el expediente aparezca como cerrado en la lista
Resultado esperado	El acta de egreso queda registrada con firma y documentos adjuntos, la cama se libera en el tablero, el expediente se marca como cerrado y el residente ya no aparece como activo
Estado	Pendiente

E2E-04: Registro de bitácora diaria con servicios e incidencia

Campo	Descripción
Identificador	E2E-04
Nombre del escenario	Registro completo de bitácora diaria
Actor principal	Encargado de turno

Precondiciones	Existen residentes activos con cama asignada. Hay turnos Matutino y Nocturno configurados. El comedor
-----------------------	---

	y los acompañamientos están registrados
Pasos a seguir	1. Iniciar sesión con rol encargado_turno. 2. Navegar a Bitácora > Nueva entrada. 3. Seleccionar el turno actual. 4. Seleccionar residentes que consumieron servicios (desayuno, comida, cena). 5. Marcar los acompañamientos correspondientes. 6. Seleccionar el comedor utilizado. 7. Registrar una visita recibida por un residente (datos del visitante, hora). 8. Registrar una incidencia (tipo, descripción, residente involucrado). 9. Guardar la entrada de bitácora. 10. Verificar que los registros aparezcan en la lista de bitácora del día
Resultado esperado	La bitácora del día queda registrada con todos los servicios prestados, la visita y la incidencia. Los datos son visibles en la consulta de bitácora
Estado	Pendiente

Matriz de Cobertura de Pruebas

Requerimiento	Descripción	Prueba Unitaria	Integración	E2E
RF-01	Autenticación y control de acceso por roles	PU-03	PI-01	E2E-01
RF-02	Gestión de catálogos y datos maestros	—	—	—

RF-03	Gestión de empleados y creación de usuarios	—	PI-02	—
RF-04	Registro de personas y apertura de expedientes	PU-02, PU-07	PI-03	E2E-01

RF-05	Gestió n de estanci as y asigna ción de camas	PU-01, PU-05	PI-03	E2E-01
RF-06	Bitácora diaria, visitas e incidenc ias	—	—	E2E-04
RF-07	Módulo clínico: cédula, plan, signos vitales	—	PI-04	E2E-02

RF-08	Valoraciones por área (Trabajo Social, Enfermería, Psicología)	—	PI-04	E2E-02
RF-09	Evaluaciones trimestrales y egreso	—	PI-04	E2E-03
RF-10	Inventario de medicamentos y administración	PU-04, PU-06	PI-05	—
RF-11	Inventario de insumos, recepciones y requisiciones	—	—	—
RF-12	Listas de verificación e inspecciones por turno	—	—	—

RF-13	Sistema de notificaciones y alertas	—	—	—
-------	-------------------------------------	---	---	---

RF-14	Auditoría y trazabilidad	—	—	—
RF-15	Adjuntos y documentos	—	—	—

Criterios de Aceptación

Una funcionalidad será considerada **aprobada** cuando cumpla con todos los siguientes criterios:

Funcionamiento correcto

- La funcionalidad ejecuta la operación solicitada sin errores de ejecución.
- Los datos se procesan y almacenan correctamente en la base de datos.
- La respuesta del sistema es consistente con la acción realizada.

Cumplimiento del requerimiento

- La funcionalidad implementa el comportamiento descrito en el requerimiento funcional correspondiente (SRS por fases).
- No existen desviaciones no autorizadas respecto a la especificación.

Manejo adecuado de errores

- Los errores de aplicación se manejan con mensajes claros y comprensibles para el usuario.
- No se expone información técnica sensible al usuario final.
- Las excepciones se registran en el log de la aplicación.
- El sistema no cae en estado inconsistente ante errores.

Validación de datos

- Los campos obligatorios se validan en el lado del cliente (JavaScript/Alpine.js) y del servidor (Laravel Request/Validation).
- Los datos inválidos son rechazados con mensajes descriptivos.
- Se respetan las restricciones de tipo, longitud y formato de cada campo.
- Las validaciones únicas (como CURP) se verifican correctamente.

Integración correcta con otros módulos

- Las operaciones que involucran múltiples módulos se ejecutan en una transacción consistente.
- Los cambios en un módulo se reflejan correctamente en los módulos dependientes.
- No se producen duplicidades ni pérdida de datos al cruzar módulos.

Ausencia de errores críticos

- No existen errores que impidan el uso de la funcionalidad.
- No hay errores de seguridad (inyección SQL, XSS, CSRF).
- No se producen errores de base de datos (violaciones de integridad, claves foráneas).
- No hay errores de lógica que produzcan datos inconsistentes.

Control de acceso

- Solo los usuarios con el rol autorizado pueden ejecutar la acción.
- Los usuarios sin permiso reciben respuesta HTTP 403.
- Las operaciones exclusivas de un rol no son accesibles desde otros roles.

Riesgos y Limitaciones

#	Riesgo / Limitación	Probabilidad	Impacto	Acción de mitigación
1	Falta de datos de prueba realistas — Los catálogos y registros de prueba pueden no reflejar la diversidad de casos reales del albergue	Alta	Medio	Utilizar Faker para generar datos variados; crear seeders con escenarios representativos (múltiples tipos de discapacidad, edades, motivos de ingreso)

2	Dependencia de servicios externos — El sistema actualmente no integra servicios externos, pero futuras canalizaciones podrían requerir APIs de instituciones gubernamentales	Baja	Alto	Diseñar la integración con interfaces abstractas ; crear mocks para servicios externos en pruebas; documentar las dependencias
---	--	------	------	--

3	Integraciones aún no desarrolladas — La Fase 8 (Inventario/Operación) y Fase 9 (Vista Móvil) están en progreso o pendientes	Alta	Alto	Priorizar pruebas de integración en los módulos completados (Fases 1-7); diferir pruebas E2E de módulos incompletos; usar datos mock para módulos
---	---	------	------	---

				pendientes
--	--	--	--	------------

4	Cambios frecuentes en los requerimientos — Los requerimientos pueden ajustarse durante el desarrollo según retroalimentación del albergue	Media	Alto	Mantener el SRS como documento vivo versionado en Git; documentar cada cambio y su impacto en pruebas existentes; ejecutar pruebas de regresión después de cada cambio
5	Tiempo limitado para realizar pruebas — El cronograma académico impone	Alta	Alto	Priorizar pruebas en los módulos críticos (admisión, clínico, egreso); automatizar pruebas

	fechas de entrega que limitan el alcance de las pruebas			unitarias con PHPUnit; documentar escenarios E2E prioritarios
6	Entorno de desarrollo con Docker/WSL — El rendimiento del entorno Docker montado desde Windows puede afectar la ejecución de pruebas automatizadas	Media	Medio	Ejecutar el proyecto desde Ubuntu/WSL para mejor rendimiento; usar bases de datos de prueba separadas del entorno de desarrollo

7	Conocimiento limitado de herramientas de testing — El equipo puede tener poca experiencia con PHPUnit o	Media	Medio	Consultar la documentación oficial de Laravel Testing; empezar con pruebas unitarias simples y escalar complejidad
---	---	-------	-------	--

	Pest			; revisar ejemplos en el directorio tests/ del proyecto
--	------	--	--	--

8	Base de datos compartida entre desarrollo y pruebas — Las pruebas podrían afectar los datos de desarrollo si no se usa una base separada	Media	Alto	Configurar una base de datos de prueba independiente en phpunit.xml ; usar transacciones reversibles en pruebas de integración; nunca ejecutar pruebas contra la base de producción
---	--	-------	------	--

Conclusión

El desarrollo del Sistema de Gestión Institucional del Albergue "Ajo'olnaj — Tu Techo" representa una oportunidad significativa para transformar la operación de la institución desde procesos manuales y desintegrados hacia una solución tecnológica coherente, segura y orientada al usuario. A través de este proyecto, el albergue podrá:

En lo operativo: Centralizar la información de todas las personas usuarias, automatizar el control de estancias y permanencias, gestionar inventario de medicamentos e insumos con trazabilidad completa, y generar reportes estadísticos confiables para la toma de decisiones.

En lo clínico: Implementar un flujo interdisciplinario estructurado que integre valoraciones de Trabajo Social, Enfermería y Psicología en una Cédula Clínica Unificada, facilitando el seguimiento mensual, las evaluaciones trimestrales y el egreso ordenado de los residentes.

En lo institucional: Fortalecer la transparencia, la trazabilidad administrativa, la protección de datos personales y el cumplimiento de protocolos y reglas de negocio establecidas, contribuyendo a la calidad del servicio y a la dignidad en la atención.

La metodología híbrida Cascada-Kanban proporciona una estructura ordenada de desarrollo por fases mientras permite flexibilidad en la gestión diaria de tareas. La arquitectura modular del sistema (11 dominios independientes) asegura mantenibilidad, escalabilidad y facilita la incorporación de nuevas funcionalidades sin afectar lo existente. Las reglas de Clean Code acordadas por el equipo, la Definition of Done rigurosa y el Plan de Pruebas integral garantizan que el producto entregado sea de alta calidad y libre de deuda técnica.

Aunque el proyecto enfrenta desafíos como cambios frecuentes en requerimientos, limitaciones de tiempo y complejidad técnica inherente a sistemas clínicos, los mecanismos de comunicación establecidos, la documentación exhaustiva y las prácticas de control de versiones mitigan estos riesgos de manera efectiva.

En conclusión, este proyecto fortalecerá la capacidad del Albergue "Ajo'olnáj — Tu Techo" para cumplir su misión de atención integral a personas en situación de vulnerabilidad, mediante una plataforma tecnológica que coloca la información y los procesos al servicio de la reinserción social, la dignidad humana y la inclusión.